# 第6章
# 数　　组

中国大学MOOC

# 学习目标

- 掌握数组的定义与使用方法，学会利用数组编写程序。
- 掌握数组元素的初始化方法。
- 掌握字符数组处理字符串的技巧。
- 熟悉字符串处理函数的功能及使用方法。
- 掌握数组作为函数参数的使用方法。

# 6 数组

- 在需要处理一批具有相同类型而且又有密切关联的数据时，数组是一个非常有用的可构造的数据类型(派生数据类型)。
- 派生数据类型：由已有的基本数据类型（char, float, double, …)或已构造的数据类型构造出的新类型。

| Grades | Codes | Prices |
|---|---|---|
| 98 | x | 10.96 |
| 87 | a | 6.43 |
| 92 | m | 2.58 |
| 79 | n | .86 |
| 85 | | 12.27 |
| | | 6.39 |

# 6.1.1 一维数组

整型表达式，表达式中可以使用符号常量。

**格式**　类型说明符 数组名[常量表达式]

**举例**

```
int  a[5];
```

**说明**

1. 数组名的命名规则应遵循标识符的命名规则。
2. 数组名后是方括号，而非圆括号。
3. 常量表达式表示元素的个数，即数组长度。

从a[0]开始，没有a[5]

```
a[0]，  a[1]，  a[2]，  a[3]，  a[4]
```

4. 常量表达式中可以包含常量和符号常量，不能包含变量。

```
#define  n  10
main()
{int  a[n], b[n+10],C[2+3];
……
}
```

Dev C++（GCC）中数组大小可用变量，例如：

```
int i=1;
int a[n];
```

# 6.1.1 一维数组的定义

| Grades | Codes | Prices |
|---|---|---|
| 98 | x | 10.96 |
| 87 | a | 6.43 |
| 92 | m | 2.58 |
| 79 | n | .86 |
| 85 | | 12.27 |
| | | 6.39 |

```
int grades[5];

char codes[4];

double price[6];
```

单选题 1分

设置

下面哪一个数组说明语句在VC 下是错误的？

A int a[]={1,2};

B char a[3];

C const int n=3; int a[n];

D int n, a[n];

提交

中国大学MOOC

# 6.1.1 一维数组的定义

- 数组在计算机内存中的存储：

  在一块连续的内存区域中存放，空间的大小为 n * sizeof（<元素类型>）。

例如： int a[5];




假定数组的第一个元素 a[0] 在内存的地址是2000, 每个元素用四个字节存贮，则



# 6.1.2 一维数组的初始化

- 一维数组的初始化：
  - 在定义数组的同时给数组元素赋初值称为数组的初始化。其语法格式为：

    类型 数组名[常量表达式]={初值1, 初值2, …, 初值n}
  - C++11 允许省略花括号之前的等号，所谓统一初始化。

    int a[10]{ 1,2,3,4,5,6,7,8,9,10 };

# 6.1.2 一维数组的初始化

1. 定义数组时，对数组元素赋初值。

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

2. 可以只给一部分数组元素赋初值。

```
int a[10]={0,1,2,3,4};
```

3. 要想使数组中全部元素数据为0，则采用如下形式

```
int a[10]={0,0,0,0,0,0,0,0,0,0};
```

4. 在对全部数组元素赋初值时，可以不指定数组长度。

```
int a[ ]={0,1,2,3,4};
```

5. 全局或静态数组的初始值默认为 0 。

其它？

可以使用：
int a[10]={0};

单选题 1分

设置

下面哪一个语句编译会告警？

A int a[] { 1, 2, 3, 4 };

B float b[]{ 1.1f, 2.2f, 3.3f };

C double c[]{ 3, 4.3, 5.5, 6 };

D char cc[] { 128, 127, 126 };

提交

中国大学MOOC

单选题 1分

设置

假设有：int a[10]={9,1,6,0,8}; 数组元素a[5]的值是：

A 8

B 0

C 6

D 随机值

提交

中国大学MOOC

# 6.1.3 一维数组的访问

格式　数组名[下标]

说明

1. 数组必须先定义，后使用。
2. 数组元素只能逐个引用，而不能一次引用整个数组。
   数组元素的引用与同类型的一般变量使用方式一样。

数组作为函数的参数时传递的是首地址。

```
……
int a[10];
a[0]=10;  a[2]=a[0]*5;
cin>>a[5];  cout<<a[5];
……
```

3. 当逐个使用数组中的每一个元素时，通常借助循环语句。

```
……
int i,a[10];
for (i=0; i<=9; i++) a[i]=i;
for (i=9; i>=0; i--) cout<<setw(2)<<a[i];
……
```



9 8 7 6 5 4 3 2 1 0

4. C++不做下标取值的合法性检查。　a[10]=1; ?

# 6.1.4基于范围for语句*

- 功能：对一个数组或STL容器(如vector)中每个元素从头到尾遍历一次。C11
- 特点：不需要循环下标。限制：数组作用域

```
int x[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
 for( int y : x ) {
    cout << y << " ";
 }
 cout << endl;
 for( auto y : x ) {
    cout << y << " ";
 }
 cout << endl;
```

单选题 1分

设置

假设有float af[] = { 1.1f, 2.2f, 3.3f, 4.4f, 5.5f };下面哪一条语句不能正确输出af中的数据？

A for (double e : af) { cout << e << " "; }

B for (auto e : af) { cout << e << " "; }

C for (float &e : af) { cout << e << " "; }

D for (int &e : af) { cout << e << " "; }

提交

中国大学MOOC

# 6.1.5 一维数组的应用

计算一元n次多项式的值

$$y = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_2 x^2 + a_1 x + a_0$$

例如： $y = 5x^5 - 3.2x^3 + 2x^2 + 6.2x - 8$

如果按多项式直接计算，会有多次重复乘法计算。

背景知识：浮点数乘法需要更多的计算周期。

可分解为n个一次式来计算：

$y = ((((5x+0)x-3.2)x+2)x+6.2)x-8$

可用一维数组来存放n+1个系数常量，然后循环n次，每次仅做一次乘法。

```
using namespace std;
#include <iostream>
int main(void){
    const int n = 5;
    float a[n+1] = {5, 0, -3.2, 2, 6.2, -8}, y = a[0], x;
    cout<<"input x=";
    cin>>x;
    for(int i = 0; i < n; i++)
        y = y * x + a[i+1];
    cout<<"y="<<y<<endl;
    return 0;
}
```

投票 最多可选1项

设置

# 最大值

问题描述

先输入一个正整数n(1到20之间)，再输入n个整数，计算其中的最大值和相应下标。

输入描述

先输入要处理的整数的个数n(1<=n<=20)，再输入n个整数，并用空格分开。

输出描述

依次输出n个整数中的最大值和相应下标（若多个整数与最大值相同，则从小到大输出相应下标），之间用空格分隔。最后不要有多余的空格。

输入样例1

5

4 6 7 2 5

输出样例1

7 2

输入样例2

5

4 6 7 2 7

输出样例2

7 2 4

A 通过测试

B 未通过测试

提交

中国大学MOOC

# 6.1.5 一维数组的应用

例：将下列 10 个数按由小到大的顺序排列输出。

15，8，0，-6，2，39，-53，12，10，6

- 选择排序
  - 第 1 次：在第 1 到第 n 个数中选出最小的，换到第 1 个位置；
  - 第 2 次：在第 2 到第 n 个数中选出最小的，换到第 2 个位置；
  - 第 i 次：在第 i 到第 n 个数中选出最小的，换到第 i 个位置；
  - i=1, 2, …, n-1 共 n-1 次。

假设a[i]最小，记为 min=a[i]，记住位置 k=i;
用a[i+1]与 min 比较，若a[i+1]更小，min=a[i+1], k=i+1;
…
用a[n-1]与 min 比较，若a[n-1]更小，min=a[n-1], k=n-1;

```
for (i=0; i<n-1; i++){
   在第 i 到第 n-1 个数中选出最小的;
   换到第 i 个位置;
}
```

交换 a[k] 与 a[i];

```
#include <iostream>
using namespace std;
int main(void){
        const int n = 10;
        int a[n] = {15,8,0,-6,2,39,-53,12,10,6};
        int i, j, minValue, minIndex, temp;
        for(i = 0; i < n - 1; i++){                          //外层循环控制轮次
            minValue = a[i];                                 //假设排头数据最小
            minIndex = i;
            for(j = i + 1; j < n; j++){                      //内层循环控制每轮比较次数
               if(minValue > a[j]){                          //是否找到更小的数
                        minValue = a[j];
                        minIndex = j;
                        }
            }
            if(i != minIndex){                  //是否需要交换
               temp = a[i];    a[i] = a[minIndex];    a[minIndex] = temp;
            }
    }
        for(i = 0; i < n; i++)    cout<<a[i]<<'\t';//输出排序后的数据
        cout<<endl;
}
```

# 设计一个函数实现选择排序算法

```
void selectSort(int data[], int n){
    int i, j, minValue, minIndex, temp;
    for(i = 0; i < n - 1; i++){
        minValue = data[i];
        minIndex = i;
        for(j = i + 1; j < n; j++){
            if(minValue > data[j]){
                minValue = data[j];
                minIndex = j;
            }
        }
        if(i != minIndex){
            temp = data[i];
            data[i] = data[minIndex];
            data[minIndex] = temp;
        }
    }
}
```

# 选择排序算法：辅助函数

```
void printArray(int data[], int n){
  for(int i = 0; i < n; i++)
     cout<<data[i]<<'\t';
  cout<<endl;
}
int main(){
  int a[10] = {15,8,0,-6,2,39,-53,12,10,6};
  selectSort(a, 10);
  printArray(a, 10);
}
```

# 简化的选择排序

另一种设计：比较之后立即交换

```
void selectSort2(int data[], int n){
   int i, j, temp;
   for(i = 0; i < n-1; i++)
       for(j = i + 1; j < n; j++)
            if(data[i] > data[j]){
                 temp = data[i];
                 data[i] = data[j];
                 data[j] = temp;
            }
}
```

每一轮循环都使最小值放到最前方；比较一次立即交换。最多交换的次数与比较次数相同，都是n*(n-1)/2。虽然交换次数多，但实现简单。

# 6.1.5 一维数组的应用

- 冒泡排序：
  - 第 0 次：将第 0 到第 n-1 个数两两比较，使大的下沉，小的上浮，最大数沉到第 n-1 个位置；
  - 第 1 次：将第 0 到第 n-2 个数两两比较，使大的下沉，小的上浮，最大数沉到第 n-2 个位置；
  - 第 i 次：将第 0 到第 n-i-1 个数两两比较，使大的下沉，小的上浮，最大数沉到第 n-i-1 个位置；
  - i=0, 1, …, n-2 共 n-1 次。

比较 a[0] 与 a[1]，若 a[0] 较大，两者交换；
比较 a[1] 与 a[2]，若 a[1] 较大，两者交换；
比较 a[j] 与 a[j+1]，若 a[j] 较大，两者交换；
j=0, 1, …, n-i-2

```
for (i=0; i<n-1; i++){
   将 0 到 n-i-1 个数两两比较，使大的下沉，小的上浮；
}
```

```
using namespace std;
#include <iostream>
int main(void)
{
    const int n=10;
    int a[n]={ 15, 8, 0, -6, 2, 39, -53, 12, 10, 6 };
    int i, j, temp;
    for (i=0; i<n-1; i++)
        for (j=0; j<n-i-1; j++)
            if (a[j]>a[j+1]){
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;}
    for (i=0; i<n; i++) cout<<a[i]<<'\t';
    cout<<endl;
    return 0;
}
```

# 6.1.5 一维数组的应用

例: 在有序数列{ -56，-23，0，8，10，12，26，38，65，98 }中，查找数据 38 是否存在。

两分查找：

两分查找也称折半查找。其方法是将有序数列逐次折半，并确定折半的数据位置，用待查找的数据与其比较，若相等则查找成功。否则比较两数的大小，如果待查找的数据比折半位置的数据小，那么到前半区间继续查找，否则到后半区间继续查找。

算法：

1. 将有序数列存放在数组 s 中。
2. 设置3个标记，low 指向待查区间的底部，high 指向待查区间的顶部，binary 指向折半的位置，即待查区间的中部：binary=(low+high)/2
3. 比较待查数据 x 与 s[binary] 是否相等，若相等，则查找成功并结束查找；若 x<s[binary]，说明 x 在 [low, binary]（前半区间）范围内，令 high=binary-1；若 x>s[binary]，说明 x 在 [binary, high] （后半区间）范围内，令 low=binary+1。
4. 将待查区间逐步缩小后，如果 low>high，则查找失败，结束查找；否则在新的区间 [low, high] 上重复 (2)、(3)、(4) 步骤。

```
#include <iostream.h>
void main(void)
{
      const int n =10;
      int s[n]={ -56, -23, 0, 8, 10, 12, 26, 38, 65, 98 };
      int low, high, binary, x;
      cout<<"input x=";
      cin>>x;
      low=0; high=n-1;
      binary=(low+high)/2;                          //确定折半位置
      while ( x!=s[binary] && low<=high ){
          if (x<s[binary]) high=binary-1;           //在前半区间查找
          else low=binary+1;                        //在后半区间查找
          binary=(low+high)/2;
      }
      if (low<=high)
          cout<<"查找成功! 是数组中的第"<<binary<<"个元素。 "<<endl;
      else
          cout<<"查找失败!"<<endl;
}
```

# 6.1.6调用标准算法简化编程*

- 真正的软件工程中，应尽可能调用系统提供的现成算法来简化编码，提高编码质量。
- C++提供了一组算法<algorithm>，可对一维数组进行各种计算，如排序、二分查找、遍历、线性查找、随机生成等。
- 尽管可能还不理解一些概念，如迭代器，C11的Lambda表达式，并不影响你通过实例来学习调用算法。
- P119 例6.5

```
#include<iostream>
#include<algorithm>
using namespace std;
int main() {
    int a[10];              //定义10个int元素的数组
    generate(a, a + 10, rand); //调用generate生成伪随机数
    for (auto x : a)            //输出显示
        cout << x << " ";
    cout << endl;
    sort(a, a + 10);            //调用sort升序排序
    for (auto x : a)                //输出显示
        cout << x << " ";
    cout << endl;
    if (binary_search(a, a + 10, 15724)) //调用二分查找
        cout << "15724 found" << endl;
    else
        cout << "15724 no found" << endl;
    auto it = find(a, a + 10, 15724);//调用find做线性查找
    if (it == a + 10)
        cout << "15724 no found" << endl;
    else
        cout << *it << " found by find" << endl;
    return 0;
}
```

# algorithm的排序

| 函数名 | 用法 |
| --- | --- |
| sort (first, last) | 对容器或普通数组中 [first, last) 范围内的元素进行排序，默认进行升序排序。 |
| stable_sort (first, last) | 和 sort() 函数功能相似，不同之处在于，对于 [first, last) 范围内值相同的元素，该函数不会改变它们的相对位置。 |
| partial_sort (first, middle, last) | 从 [first,last) 范围内，筛选出 muddle-first 个最小的元素并排序存放在 [first，middle) 区间中。 |
| partial_sort_copy (first, last, result_first, result_last) | 从 [first, last) 范围内筛选出 result_last-result_first 个元素排序并存储到 [result_first, result_last) 指定的范围中。 |
| is_sorted (first, last) | 检测 [first, last) 范围内是否已经排好序，默认检测是否按升序排序。 |
| is_sorted_until (first, last) | 和 is_sorted() 函数功能类似，唯一的区别在于，如果 [first, last) 范围的元素没有排好序，则该函数会返回一个指向首个不遵循排序规则的元素的迭代器。 |
| void nth_element (first, nth, last) | 找到 [first, last) 范围内按照排序规则（默认按照升序排序）应该位于第 nth 个位置处的元素，并将其放置到此位置。同时使该位置左侧的所有元素都比其存放的元素小，该位置右侧的所有元素都比其存放的元素大。 |

# 6.3 数组与函数

- 数组可以作为函数的参数使用，包括：
  - 数组元素作为函数的参数
  - 数组名作为函数的参数

  两种情况。

中国大学MOOC

## 数组元素作函数的参数

```
#include <iostream>
using namespace std;
void  change(int x, int y)
{
    int tmp;
    tmp=y;
    y=x;
    x=tmp;
    cout<<“x=“<<x<<“y=“<<y<<endl;
}
int main()
{  int  x[2]={3, 4};
    cout<<x[0]<<‘ ‘ <<x[1]<<endl;
    change(x[0], x[1]);
    cout<<x[0]<<‘ ‘ <<x[1]<<endl;
}
```

单个的数组元素同简单变量使用没有差异，凡是允许简单变量的地方，下标变量同样使用

```
using namespace std;
#include <iostream>
int c[10];

void add(int x, int y, int k)
{  c[k] = x+y;  }

int main(void)
{       int a[10]={1,2,3,4,5,6,7,8,9,10};
        int b[10]={10,9,8,7,6,5,4,3,2,1},i;
        for (i=0; i<10; i++) add(a[i], b[i],i);
        for (i=0; i<10; i++) cout<<c[i]<<"  ";
        cout<<'\n';
        return 0;
}
```

# 6.3 数组与函数

- 数组名作为函数的参数：
  1. 传递给函数的是数组第一个元素在计算机内存中的位置（地址），这时形参数组与实参数组共享内存中的存储空间。
  2. 若仍然用值传递的问题：占据大量的空间
  3. 调用函数时，因只将数组第一个元素的地址传递给函数，而与数组本身的数据个数无关，所以，一般在使用时，还须将元素个数作为一个参量提供。

```
using namespace std;
#include <iostream>
int main(void){
        const int n = 10;
        int a[n] = {15, 8, 0, -6, 2, 39, -53, 12, 10, 6 };
        int i, j, minValue, minIndex, temp;
        for(i = 0; i < n - 1; i++){
            minValue = a[i];
            minIndex = i;
            for(j = i + 1; j < n; j++){
                        if(minValue > a[j]){
                                    minValue = a[j];
                                    minIndex = j;
                        }
            }
            if(i != minIndex){
                        temp = a[i];
                        a[i] = a[minIndex];
                        a[minIndex] = temp;
            }
        }
        for(i = 0; i < n; i++)    cout<<a[i]<<'\t';
        cout<<endl;
        return 0;
}
```

```
using namespace std;
#include <iostream>
#include <ctime>
#define NUM 20
void selectSort(int data[], int n){
    int i, j, minValue, minIndex, temp;
    for(i = 0; i < n-1; i++){
        minValue = data[i];  minIndex = i;
        for(j = i + 1; j < n; j++){
            if(minValue > data[j]){
                minValue = data[j];
                minIndex = j;
            }
        }
        if(i != minIndex){
            temp = data[i];
            data[i] = data[minIndex];
            data[minIndex] = temp;
        }
    }
}
```

```
void printarr(int a[],int n)
{
    int i;
    for(i=0;i<n;i++)
        cout<<a[i]<<' ';
    cout<<endl;
}

int main()
{
    int x[NUM];
    int i;
    srand((unsigned)time(NULL));
     for(i=0;i<NUM;i++)
            x[i]=rand()%100;
     printarr(x,NUM);
     selectSort(x,NUM);
     printarr(x,NUM);
     return 0;
```

main的后三句常写成如下格式：

```
printarr(x,sizeof(x)/sizeof(int));
selectsort(x,sizeof(x)/sizeof(int));
printarr(x,sizeof(x)/sizeof(int));
```

# 6.3 数组与函数

```
for(i=0;i<n-1;i++)
  for(j=i+1;j<=n-1;j++)
     if( a[j]<a[i] )   a[j]与a[i]互换;
```

⟹

```
for(i=0;i<n-1;i++)
   for(j=i+1;j<=n-1;j++)
     if( a[j]<a[i] )  { t=a[i];
                        a[i]=a[j];
                        a[j]=t;}
```

```
 void  selectsort(int a[],int n)
 {
  int i,j,t;
   for(i=0;i<n-1;i++)
    for(j=i+1;j<=n-1;j++)
      if( a[j]<a[i]){ t=a[i];
                      a[i]=a[j];
                      a[j]=t;
                    }
 }
```

```
int main()
{
        int x[NUM];
        int i;
        srand((unsigned)time(NULL));
        for(i=0;i<NUM;i++)
            x[i]=rand()%100;
        selectsort(x,NUM);
}
```

单选题 1分

设置

若函数void sort(int a[], int n);实现大小n的数组a的排序。
设有语句： int b[7];
下面哪一个函数调用能正确执行？

A sort(b);

B sort(b, 9);

C sort(b, sizeof(b)/sizeof(int))

D b = sort(b, 7);

提交

中国大学MOOC

投票 最多可选1项

设置

# 最大值（函数）

问题描述

先输入一个正整数n(1到20之间)，再输入n个整数，计算其中的最大值和相应下标。利用函数int findmax(int a[],int n)获得数组a中的最大值，在main中调用findmax实现输入、函数调用、输出等操作。

输入描述

先输入要处理的整数的个数n(1<=n<=20)，再输入n个整数，并用空格分开。

输出描述

依次输出n个整数中的最大值和相应下标（若多个整数与最大值相同，则从小到大输出相应下标），之间用空格分隔。最后不要有多余的空格。

输入样例1

5

4 6 7 2 5

输出样例1

7 2

输入样例2

5

4 6 7 2 7

输出样例2

7 2 4

A 通过测试

B 未通过测试

提交

# 6.2 二维数组

- 例如：三个城市各季节的平均气温

| | 春 | 夏 | 秋 | 冬 |
|---|---|---|---|---|
| 北京 | 11.3 | 28.2 | 16.5 | -7.1 |
| 天津 | 12.4 | 27.1 | 17.6 | -5.7 |
| 上海 | 23.2 | 33.5 | 25.8 | -1.3 |

是一个 3×4 矩阵，可用二维数组存贮表示。

中国大学MOOC

# 6.2.1 二维数组的定义

**格式**　类型说明符 数组名[常量表达式1] [常量表达式2]

**举例**

```
float  a[2][3], b[4][5];
```

**说明**

1。二维数组中元素的排列顺序是：先行后列。
因此，可以把二维数组看成是一个矩阵。

2。二维数组元素仍然是从a[0][0]开始。

```
a[0][0]，  a[0][1]，  a[0][2] ……
a[1][0]，  a[1][1]，  a[1][2] ……
……
```

中国大学MOOC

# 6.2.1 二维数组的定义

- 定义二维数组如下：

  **float T[3][4]；**

- 这个数组在内存中占用 12 个连续的 float 元素的存储单元。
- 二维数组元素用两个下标表示，分别表示所在的行和列。

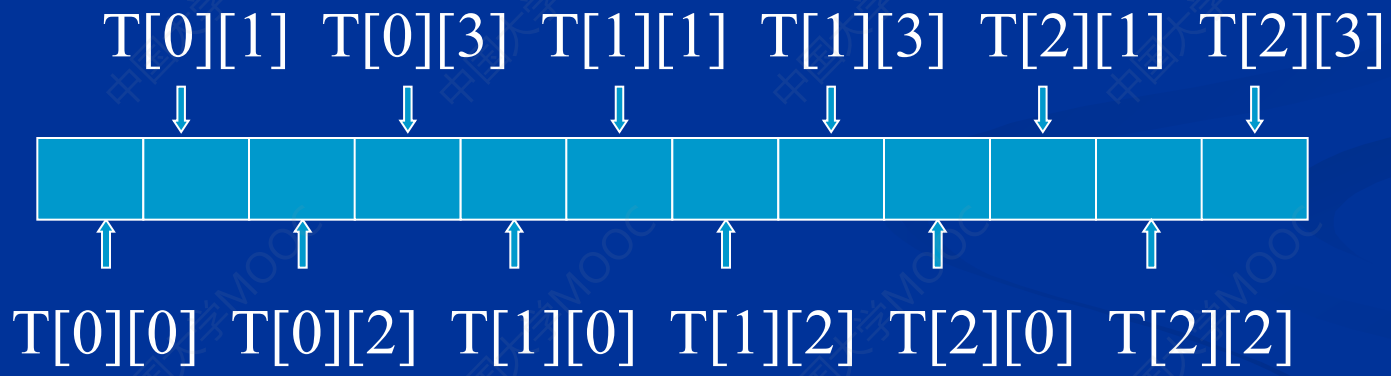
  **T[0][0], T[0][1], T[0][2], T[0][3],**

  **T[1][0], T[1][1], T[1][2], T[1][3],**

  **T[2][0], T[2][1], T[2][2], T[2][3]。**

# 6.2.1 二维数组的定义

- 二维数组在内存中的存放仍然是一维的，而且是按行顺序存放的，即在内存中先放第一行的元素，再放第二行的元素，…



# 6.2.1 二维数组的定义

- C++语言把二维数组看成是一个一维数组，该一维数组的各个元素又是一个一维数组。例如：

T { T[0]——T[0][0], T[0][1], T[0][2], T[0][3],
  { T[1]——T[1][0], T[1][1], T[1][2], T[1][3],
  { T[2]——T[2][0], T[2][1], T[2][2], T[2][3],

- 由这种形式我们可以了解更高维的数组。一个三维数组实际上可以看作是元素为二维数组的数组，依此类推。

# 6.2.2 二维数组的初始化

- 二维数组的初始化：

1. 分行给二维数组赋初值。

```
int a[2][3]={{1,2,3},{4,5,6}};
```

1 2 3
4 5 6

2. 按数组的排列顺序对各数组元素赋初值。

```
int b[2][3]={1,2,3,4,5,
```

1 2 3
4 5 6

3. 可以对部分元素赋初值。

```
int c[3][4]={{1},{5},{9}};
```

1 0 0 0
5 0 0 0
9 0 0 0

```
int d[3][4]={{1},{5,6},{0,
```

1 0 0 0
5 6 0 0
0 9 7 0

4. 在对全部数组元素赋初值时，数 长度可以不指定。

```
int e[ ][3]={1,2,3,4,5,6};
```

1 2 3
4 5 6

```
int f[ ][4]={{0,0,3},{0},{0,10}};
```

0 0 3 0
0 0 0 0
0 10 0 0

单选题 1分

设置

下面哪一个数组说明语句是错误的？

A int b[][3]={0,1,2,3};

B int d[3][ ]={{1,2},{1,2,3},{1,2,3,4}};

C int c[100][100]={0};

D int a[2][3];

提交

单选题 1分

设置

有语句：int b[][3]={{9},{1,6},{0,8},{1,2,3}}; 数组元素b[3][2]的值是:

A 3

B 8

C 6

D 9

提交

# 6.2.2 二维数组的应用

格式　数组名[下标1] [下标2]

说明　1。数组必须先定义，后使用。

用，而不

类型的一

```
float  a[5][6];
for (i=1; i<=4; i++)
   for (j=1; j<=5; j++)
      cin>>a[i][j];
```

```
float  a[5][6];
for (i=1; i<=4; i++)
   {for (j=1; j<=5; j++)
      cout<<a[i][j];
    cout<<endl;}
```

```
a[0][1]=10; a[0][2]=a[0][1]
cin>>a[1][1]; cout<<a[1][1];
.....
```

3.二维数组输入/输出的一般格式。

```
.....
int i,j;
float  a[4][5];
for (i=0; i<=3; i++)
   for (j=0; j<=4; j++)
      cin>>a[i][j];
.....
```

```
.....
int i,j;
float  a[4][5];
for (i=0; i<=3; i++)
  { for (j=0; j<=4; j++)
      cout<<a[i][j];
    cout<<endl;}
.....
```

单选题 1分

设置

执行以下代码段后，t 的值为：

```
int b[3][3]={0,1,2,0,1,2,0,1,2},t=1;
for(int i=0;i<3;i++)
  for(int j=i;j<=i;j++)
    t = t + b[i][i] + b[j][j];
```

A 7

B 9

C 4

D 3

提交

中国大学MOOC

# 6.2.2 二维数组的应用

- 二维数组的应用：

  编程实现两个矩阵的乘法运算。

  根据矩阵乘法的运算规律：$M_{n0\times n1}\times N_{n1\times n2}=Q_{n0\times n2}$

例如：

$$\begin{bmatrix}1&2&3&4\\2&2&3&1\\5&4&2&3\end{bmatrix}\times\begin{bmatrix}6&3&2\\2&8&1\\6&9&5\\2&4&6\end{bmatrix}=\begin{bmatrix}36&62&43\\36&53&27\\56&77&42\end{bmatrix}$$

其中：

$$q[i][j]=\sum_{k=1}^{n1} m[i][k]\times n[k][j]$$

```
#include <iostream>
using namespace std;
int main(void)
{
    int m[3][4]={{1,2,3,4}, {2,2,3,1}, {5,4,2,3}};
    int n[4][3]={{6,3,2}, {2,8,1}, {6,9,5}, {2,4,6}};
    int i, j, k, q[3][3]={0};
    for (i=0; i<3; i++)
        for (j=0; j<3; j++)
            for (k=0; k<4; k++)
                q[i][j]=q[i][j]+m[i][k]*n[k][j];
    for (i=0; i<3; i++){
        for (j=0; j<3; j++) cout<<q[i][j]<<"\t";
        cout<<endl;
    }
    return 0;
}
```

投票 最多可选1项

设置

# 对角线交换

问题描述

将一个n*n矩阵元素根据对角线交换。

输入描述

第一行输入一个整数n表示矩阵的大小，接下来输入n行,每行有n个整数(1 <= n <= 500)。

输出描述

输出根据对角线交换后的n*n矩阵,输出格式如下: 输出n行,每行输出n个整数。每两个整数之间都有一个空格,记住,每行最后一个整数后面没有空格。

输入样例

4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

输出样例

1 5 9 13
2 6 10 14
3 7 11 15
4 8 12 16

A 自行测试通过

B 未通过测试

提交

# 一年中的第几天

**题目内容：**
计算某年某月某日是该年中的第几天。
**输入格式:**
依次输入年、月和日
**输出格式：**
输出这一天是该年中的第几天。注意年月日合法性的判定和闰年的计算

**输入样例1：**
2009 3 2
**输出样例1：**
61

**输入样例2：**
2009 3 32
**输出样例2：**
日期有误

```
#include <iostream>
using namespace std;
int main(){
        int days[12]={0,31,59,90,120,151,181,212,243,273,304,334};
        int year,month,day;
        bool leap;
        cin>>year>>month>>day;
        if(year<0||month<0||month>12||day<0||month!=2&&day>(days[month]-
days[month-1]+1)) {
            cout<<"日期有误";
            return 0;
        }
        leap=year%400==0||(year%4==0&&year%100!=0);
        if(month==2){
            if(day>29||!leap&&day>28){    cout<<“日期有误”;    return 0;    }
        }
        if(leap)
            cout<<(month<=2?days[month-1]+day:days[month-1]+day+1);
        else
            cout<<days[month-1]+day;
        return 0;
}
```

# n以内的素数

**题目内容：**
输出n以内的所有素数
**输入格式:**
输入一个不大于1000的正整数n
**输出格式：**
输出<=n的所有素数，素数之间用空格分割

**输入样例1：**
50
**输出样例1：**
3 5 7 11 13 17 19 23 29 31 37 41 43 47

```
using namespace std;
#include <iostream>
int main(){
    int m,i,n;
    cin>>n;
    for(m=3;m<=n;m=m+2){
        for(i=2;i*i<=m;i++)
            if(m%i==0) break;
        if(i*i>m) cout<<m<<' ';
    }
    return 0;
}
```

```
using namespace std;
#include <iostream>
int main(){
    int i,j,n;
    cin>>n;
    int p[1000];
    for(i=0;i<=n;i++)
        p[i]=1;
    for(i=2;i<=n;i++)
        if(p[i]==1)
            for(j=2;i*j<=n;j++) p[i*j]=0;
    for(i=2;i<=n;i++)
        if(p[i]) cout<<i<<' ';
    return 0;
}
```

vector<int> p(n+1,1);

# 升序数组的随机生成

编写函数bool insertSort(int a[],int maxnum,int n, int x)，形参a是一个数组，形参maxnum是数组a的大小，即能容纳最多元素个数，n是当前已插入的元素个数，x是要插入的值。该函数将x插入到数组a中的合适位置，使所有元素保持升序排列。如成功插入，返回true，否则（数组已满），则返回false。

编写函数void printArr(int a[],int n)，将大小为n的数组a输出。

已给出main函数，在main函数中产生100以内的随机数，并通过循环调用insertSort函数，将产生的随机数插入升序数组中，保持其升序，并最终输出产生的升序数组。

```
#define NUM 20
#include <iostream>
#include <cstdlib>
#include <ctime>
int main()
{
    int x[NUM];
    int i;
    srand((unsigned)time(NULL));
    for(i=0; insertSort(x, NUM,i, rand()%100);i++);
    printArr(x,i);
    return 0;
}
```

```
using namespace std;
#define NUM 20
#include <iostream>
#include <cstdlib>
#include <ctime>
bool insertSort(int a[], int maxnum, int n, int x)
{
      int i;
      if(maxnum<=n) return false;
      for(i=n;x<a[i-1]&&i>0;i--)
                  a[i]=a[i-1];
      a[i]=x;
      return true;
}
void printArr(int a[],int n){
      int i;
      for(i=0;i<n;i++)
         cout<<a[i]<<'\t';
      cout<<endl;
}
```

```
int main()
{
   int x[NUM];
   int i;
   srand((unsigned)time(NULL));
   for(i=0; insertSort(x, NUM,i, rand()%100);i++);
   printArr(x,i);
   return 0;
}
```

# 6.4 容器vector与map*

- 如果需要一个数组在运行时大小能动态伸缩，就应选用一种STL序列容器。
- vector最简单最常用。
  - 一个vector管理一组元素，加入和删除元素适合在尾端操作
  - 在定义vector变量之前应先包含#include<vector>。
- map的下标不局限于从0开始。

# 6.4.1 vector向量

定义一个向量有多种形式：

- vector<元素类型> 变量名;
- vector<元素类型> 变量名(元素个数);
- vector<元素类型> 变量名(元素个数, 元素初值);
- vector<元素类型> 变量名(变量2);
- vector<元素类型> 变量名(begin, end);

```
vector<int> first;//创建一个值类型为int的空vector

vector<int> second(10);//创建一个10个int的vector，且每个元素初值为0

vector<int> third(10,1);//创建一个10个int的vector，且每个元素初值为1

vector<int> forth(third);//将vector third 拷贝给vector forth ，两者元素值完全相同

vector<int> fifth(third.begin(),third.end()-5);
//将third [third.begin(),third.end()-5)区间的元素赋给fifth

//从数组中获得初值
int b[7]={1,2,3,4,5,6,7};
vector<int> a(b,b+7);
```

# 6.4.1 vector向量

常用成员函数：

- ✓ void push_back(元素)　将指定元素添加到尾端 void
- ✓ pop_back()　删除尾端一个元素
- ✓ size_t size()　获取当前元素个数
- ✓ iterator begin()　获取头一个元素的位置
- ✓ iterator end()　获取最后元素之后的空位置
- ✓ iterator insert(pos, 元素)　在指定位置插入指定元素，并返回迭代器指向该元素
- ✓ iterator erase(pos)　取出并删除确定位置上的元素，后面元素前移

```
#include<iostream>
#include<vector>
#include<algorithm>
#include<ctime>
using namespace std;
void print(vector<int>& v) {
	cout << "Total " << v.size() << " elems:" << endl;
	for (auto x : v)
		cout << x << " ";
	cout << endl;
}
int main() {
	vector<int> v1(10); //定义一个vector有10个int元素
	srand((unsigned)time(NULL));
	generate(v1.begin(),v1.end(),[]{return rand()%100+1;}); //产生1-100随机数
	print(v1);
	sort(v1.begin(), v1.end(), [](auto x1, auto x2){return x1 < x2;});//排序
	print(v1);
	const int wanted = 65;
	if (binary_search(v1.begin(), v1.end(), wanted))    //二分查找
		cout << "found " << endl;
	else
		cout << "no found"<<endl;
	v1.erase(v1.begin()+3); //删除第4个元素
	print(v1);
	v1.insert(v1.begin()+3,100); //添加数据
	print(v1);
	return 0;
}
```

Lambda 表达式

```
#include<iostream>
#include<vector>
using namespace std;
void print(vector<vector<int> >& table){
    cout<<table.size()<<endl;
    for(auto &row : table){
        int sum = 0;
        cout<<row.size()<<":";
        for(auto &elem : row){
              sum += elem;
               cout<<elem<<" ";
        }
        cout<<":avg="<<(float)sum/row.size()<<endl;
    }
}
```

```
int main(){
    int row = 0;
    cin >> row;
    vector<vector<int> > table(row);
    for (int i = 0; i < row; i++){
              int num = 0;
              cin>>num;
              for (int j = 0; j < num ; j++){
                    int k = 0;
                    cin >> k;
                    table[i].push_back(k);
              }
    }
    print(table);
    return 0;
}
```

# 6.4.2 map映射

- ✓ 一个map包含一组对偶(或有序对) pair<key, value>。
- ✓ 一个对偶表示从一个键key到一个值value的决定关系。
- ✓ 一个map容器中的所有的键key组成一个集合set，即键不重复且按升序排序。
- ✓ 数组可用map实现，如

  float a[10] => map<unsigned, float> a

- ✓ 要使用map应先包含#include<map>

P131 例6.12

# 6.4.3 初始化列表与统一初始化

C++11引入初始化列表initializer list，支持所谓的统一初始化uniform initialization。

语法特征是花括号括起一个或多个初始值，而且可省略前面的等号。

例如：

```
int a[] = {2,4,6};
int b[] {1,3,5};
vector<int> c{3,4,5};
map<int, float> d {{1, 2.0f}, {2, 3.3f}, {4, 4.4f}};
```

# 6.5.1 字符数组的定义

- 在数组元素的类型是字符时，称为字符数组
- 定义格式：

  char <数组名> [常量表达式]

- C/C++中没有字符串变量，通过字符数组来达到此功能

# 6.5.2 字符数组的初始化

- 初始化：
  - char s[10]={‘a’,’b’,’c’};
  - char x[ ]={‘a’,’b’,’c’};
  - char y[ ]={“abc”};
  - char z[ ]=“abc”;
- 字符串：以‘\0’结束的一个字符序列（0本身不表示）
- 上述例子中：x的大小是3，y,z是4

单选题 1分

设置

下面哪一个数组说明语句是正确的？

A char s4[2][3]={"xyz","abc"};

B char s1[]="xyz";

C char s3[][]={'x','y','z'};

D char s2[3]="xyz";

提交

中国大学MOOC

单选题 1分

设置

设有说明语句：char s[5][5] ={“abc”,“efgh”}; 值为字符g 的数组元素是：

A s[4][4]

B s[1][4]

C s[1][3]

D s[1][2]

提交

中国大学MOOC

# 更多的初始化形式*

UTF-8字符串可作为char窄串，前缀u8：

- char str1[] = u8"Hello World";

对char16_t和char32_t数组的宽串初始化，分别添加u和U前缀：

- char16_t strr1[] = u"hello"; //UTF-16串
- char32_t strr2[] = U"hello"; //UTF-32串

C11开始支持raw串（毛串或原始串），其中可包含任意字符，包括双引号"、反斜杠\、换行符，其中不需要转义符。raw串用R前缀：

- char raw_narrow []= R"(An unescaped \ character)";
- wchar_t raw_wide[]= LR"(An unescaped \ character)";
- char raw_utf8[] = u8R"(An unescaped \ character)";
- char16_t raw_utf16[]= uR"(An unescaped \ character)";
- char32_t raw_utf32[]= UR"(An unescaped \ character)";

# 字符串初始化的要点

- 用字符串初始化时，系统在字符串末尾自动加一个0值，作为串结尾标志。因此字符数组长度至少比串中字符个数大1。
- 系统能根据字符串长度(即字符个数)来确定数组的大小。
- 用字符串初始化一维字符数组时，大多省略花括号。
- *除了raw串，其它字符串中都可带有转义字符。
- 不含任何字符的字符串""称为空串，尾0也要占1个字符。
- 除了用字符数组定义字符串，字符串还可用字符指针来定义，或类型(如std::string)来定义。

中国大学MOOC

# 6.5.3字符数组的输入输出

1、逐个元素处理

- cin>>c与cout<<c
- cin.get()与cout.put()

2、按字符串处理

- cin>>str与cout<<str;　　cin>>不安全
- cin.getline(str, 20);　　限定长度不越界

3*、wchar_t宽串要用wcin, wcout, 而且

- wcin.imbue(locale(“chs”));　//中文简体，输入
- wcout.imbue(locale(“chs”)); //中文简体，输出
- wchar_t weekday[] = L"日一二三四五六";
- wcout.imbue(locale("chs"));
- wcout << L"星期"<<weekday[week] << endl;

# 6.5.3 字符数组的输入输出

- 字符数组中若是字符串，可直接输出

```
char s[ ]="abcd   efg";
int y[ ]={1,2,3,4};
cout<<s<<endl;   //OK
cout<<y<<endl;  //error
```

单选题 1分

设置

下列程序的输出结果是。

```
int main(void){
  char w[][10]={"ABCD","EFGH","IJKL","MNOP"}, k;
  for(k=1; k<3; k++)
     cout<<w[k];
  cout<<endl;
  return 0;
}
```

A BCD

B EFGH

C IJKL

D EFGHIJKL

提交

投票 最多可选1项

设置

# 字符串逆序数

问题描述

计算并返回字符串a的逆序。逆序的概念：在一个字符串中，如果存在i<j，且a[i]>a[j]，则称a[i]和a[j]构成一个逆序。例如"DAABEC"的逆序是5，其中D与A、A、B、C构成4个逆序，E与C构成1个逆序。

输入描述

输入一个字符串，长度不超过100。

输出描述

输出一个整数，为输入字符串的逆序数目。

输入样例

DAABEC

输出样例

5

A 测试通过

B 未通过测试

提交

中国大学MOOC

投票 最多可选1项 设置

# 字符串逆序数

问题描述

计算并返回字符串a的逆序。逆序的概念：在一个字符串中，如果存在i<j，且a[i]>a[j]，则称a[i]和a[j]构成一个逆序。例如"DAABEC"的逆序是5，其中D与A、A、B、C构成4个逆序，E与C构成1个逆序。

说明函数int getRev(char a[])返回a中字符串的逆序数目；main函数中通过输入、函数调用、输出实现问题要求。

输入描述

输入一个字符串，长度不超过100。

输出描述

输出一个整数，为输入字符串的逆序数目。

输入样例

DAABEC

输出样例

5

A 测试通过

B 未通过测试

提交

# 6.6.1 字符数组处理函数

- 常用字符数组中字符串处理函数：(用于字符串处理的函数在头文件string.h或cstring中)

  int strlen (char *s)　　求字符串长度

  char* strcpy (char *s1,char *s2)

  字符串拷贝

  char* strcat (char *s1,char *s2)

  字符串连接

  int strcmp (char *s1,char *s2)

  字符串比较

# 6.6.1 字符数组处理函数

- 其他函数：
  - memcpy(b1,b2,n) 将b2开始的n个字节拷贝到b1开始
  - memmove(b1,b2,n) 将b2开始的n个字节拷贝到b1开始(b1,b2可重合）
  - memset(b,ch,n) 将b开始的n个字节用ch填充

# 6.6.1 字符数组处理函数

```
int a[100][100],b[100][100];
for(i=0;i<100;i++)
  for(j=0;j<100;j++)
    a[i][j]=0;   //a清0
```

```
for(i=0;i<100;i++)
  for(j=0;j<100;j++)
    b[i][j]=a[i][j];   //a拷贝到b
```



```
memset(a,'\0',sizeof(a))
```

```
memcpy(b,a,sizeof(a))
```

---

```
int x[]={1,2,3,4,5};
memmove(x,x+1,sizeof(x)-sizeof(int))
```

结果： x: 2 ,3, 4, 5, 5

# 实现strcat函数，并在主函数中测试。

```
using namespace std;
#include <iostream>
void strcat(char t[],char s[])
{
  int i,j;
  i=j=0;
  while(t[i]!='\0') i++;
  while(s[j]!='\0')
    { t[i]=s[j];
      i++;
      j++;
    }
  t[i]='\0';
}

int main()
{
  char a[40]={"abcdef"};
  strcat(a,"xyz");
  cout<<a<<endl;
  return 0;
}
```

```
using namespace std;
#include <iostream>
void strcat(char t[],char s[])
{
   int i,j;
   i=j=0;
   while(t[i++]) ;
   i--;
   while(s[j])
     t[i++]=s[j++];
   t[i]='\0';
}

int main()
{
  char a[40]={"abcdef"};
  strcat(a,"xyz");
  cout<<a<<endl;
  return 0;
}
```

```
using namespace std;
#include <iostream>
void strcat(char t[],char s[])
{
   int i,j;
   i=j=0;
   while(t[i++]) ;
   i--;
   while(t[i++]=s[j++]);
}

int main()
{
  char a[40]={"abcdef"};
  strcat(a,"xyz");
  cout<<a<<endl;
  return 0;
}
```

# 例：输入若干数组串，升序排序后输出。

```
using namespace std;
#include <iostream>
#include <cstring>

void sortString(char str[][20],int n){
    int i,j;
    char tmp[20];
    for(i=0;i<n-1;i++)
      for(j=i+1;j<=n-1;j++)
       if(strcmp(str[i],str[j])>0){
              strcpy(tmp,str[i]);
              strcpy(str[i],str[j]);
              strcpy(str[j],tmp);
        }
}
```

```
void inputString(char str[][20],int n){
    int i;
    for(i=0;i<n;i++)
      cin>>str[i];
}
void printString(char str[][20],int n){
    int i;
    for(i=0;i<n;i++)
      cout<<str[i]<<endl;
}

int main() {
    char str[3][20];
    cout<<"Input strings:"<<endl;
    inputString(str,3);
    sortString(str,3);
    cout<<"Ascending strings:"<<endl;
    printString(str,3);
}
```

投票 最多可选1项 设置

# 最长的字符串

问题描述
输入若干个字符串，将最长的字符串输出。如有多个字符串长度相同，则输出第一个最长的字符串。
说明函数void LongestStr(char a[][100],int n)输出a中n个字符串中第一个最长字符串； void InputStr(char a[][100],int n) 实现n个字符串的输入；main函数中通过数组声明、函数调用等实现问题要求。
输入描述
输入一个整数n(1<=n<=10)，然后输入n个字符串，长度不超过100。
输出描述
输出所输入的字符串中最长的字符串。
输入样例
3
ABC
NJUST
China
输出样例
NJUST

A 自行测试通过

B 未通过测试

提交

# 6.6.2 string类型*

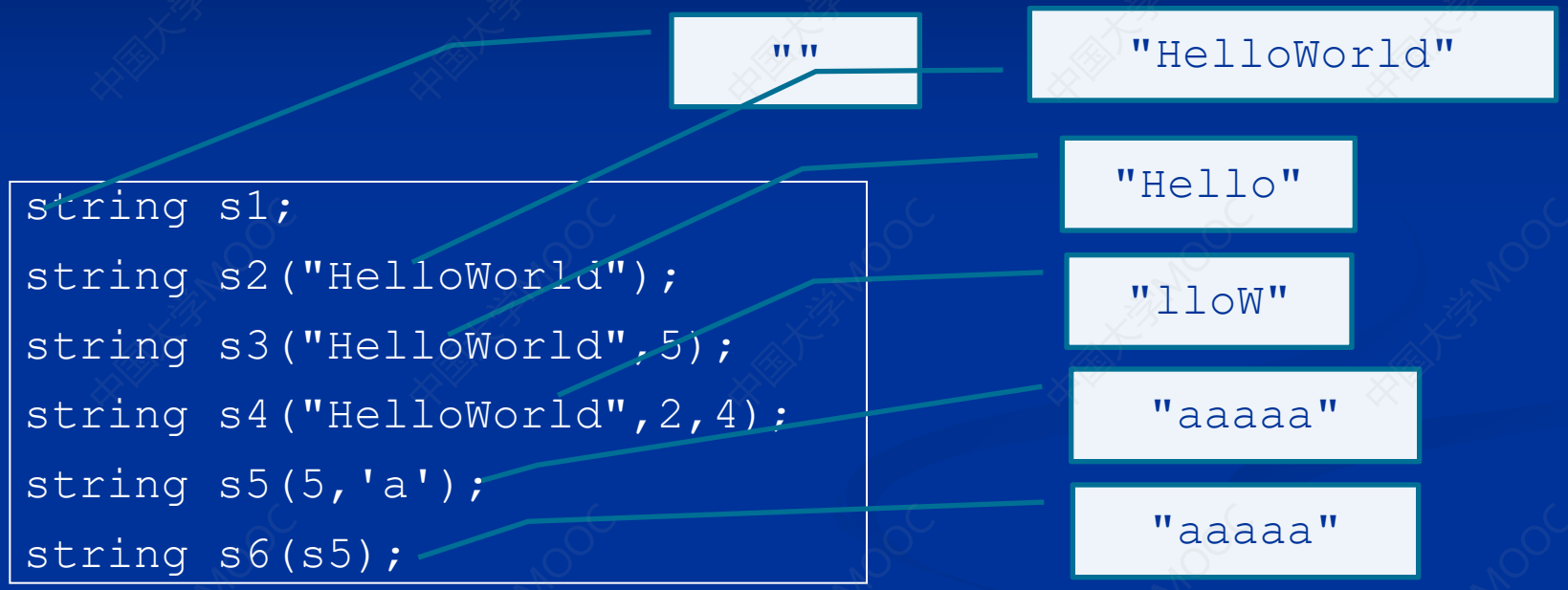
如果需要长度可动态伸缩的、操作安全的字符串，就需要<string>的string类型。
详见p129-142

| 串类型名称 | 中文名称 | 字符类型 | 输出 | 输入 |
| --- | --- | --- | --- | --- |
| string | 串、窄串 | char | cout | cin |
| wstring | 宽串 | wchar_t | wcout | wcin |
| u16string | 小u串 | char16_t | 不支持 | 不支持 |
| u32string | 大U串 | char32_t | 不支持 | 不支持 |

# string类型

- string类型是长度可动态伸缩、操作安全的字符串，在<string>头文件中定义。

# string串的创建


""
"HelloWorld"
"Hello"
"lloW"
"aaaaa"
"aaaaa"
string s1;
string s2("HelloWorld");
string s3("HelloWorld",5);
string s4("HelloWorld",2,4);
string s5(5,'a');
string s6(s5);

# string串指定元素的访问

- 可用下标访问串s中指定元素，下标范围从0到s.size()-1。

```
using namespace std;
#include <iostream>
#include <string>
int main(){
  string s("Hello");
  for(auto &c:s)
      c=toupper(c);
  for(auto k:s)
      cout<<k;
  }
  return 0;
} ;
```

```
HELLO
```

# string串上的运算

- 赋值运算符=，字符串拷贝。
- 关系运算符<、<=、>、>=、==、!=，字符串比较
- 加法运算符+、+=，字符串拼接

s1: World

s2: HelloWorld

```
using namespace std;
#include <iostream>
#include <string>
int main(){
  string s1,s2;
  s1="World";
  s2="Hello"+s1;
  if(s1>s2)
    cout<<s1;
  else cout<<s2;
  return 0;
} ;
```

```
World
```

# string串的输入

```
World
```

- string s;
- cin>>s; //空格、tab为分隔符
- getline(cin,s); //空格、tab为普通字符

投票 最多可选1项 设置

# 最长的字符串

问题描述
输入若干个字符串，将最长的字符串输出。如有多个字符串长度相同，则输出第一个最长的字符串。
说明函数void LongestStr(string a[],int n)输出a中n个字符串中第一个最长字符串； void InputStr(string a[],int n) 实现n个字符串的输入；main函数中通过数组声明、函数调用等实现问题要求。
输入描述
输入一个整数n(1<=n<=10)，然后输入n个字符串，长度不超过100。
输出描述
输出所输入的n个字符串中最长的字符串。
输入样例
3
ABC
NJUST
China
输出样例
NJUST

A 自行测试通过

B 未通过测试

提交

# 一年中的第几天

**题目内容：**
计算某年某月某日是该年中的第几天。
**输入格式:**
输入字符串，格式为：date:年-月-日
**输出格式：**
输出这一天是该年中的第几天。注意年月日合法性的判定和闰年的计算

**输入样例1：**
date:2009-3-2
**输出样例1：**
61

```
scanf("date:%d-%d-%d",&year,&month,&day);
```

**输入样例2：**
date:2009-3-32
**输出样例2：**
日期有误

```
int year,month,day;
char d[20];
int i;
//scanf("date:%d-%d-%d",&year,&month,&day);
cin>>d;
for(i=5,year=0;d[i]!='-';i++)
        year=year*10+d[i]-'0';
for(i++,month=0;d[i]!='-';i++)
        month=month*10+d[i]-'0';
for(i++,day=0;d[i];i++)
      day=day*10+d[i]-'0';
```

编程实现输入若干个字符串，将字符串中大写字符全部转成小写字符后，将字符串中子串排序后，将字符串输出。

要求：

1）编写函数：void upp_to_low(char a[])，将数组a中的字符串中大小字母全部转成小写。例如，a中有字符串“I love NJUST”，转为小写后，a中字符串为“i love njust”。

2）编写函数：void sort_substr(char a[])，将数组a中的字符串中子串排序。例如，a中有字符串"i love njust"，则有3个子串i、love和njust，分别将这三个子串按字母序升序排序后为i、elov和jnstu，则a中的新串为"i elov jnstu"。

3）在main中先输入n(1<=n<=20)，然后输入n个包含空格的字符串，每个串不多于99个字符，循环调用upp_to_low函数和sort_substr函数，完成对n个字符串的字母大小写转换和对子串的排序后，输出排序后的n个字符串。

```
#include <iostream>;
using namespace std;
void upp_to_low(char a[]){
      for(int i=0;a[i];i++)
            if(a[i]>'A'&&a[i]<'Z') a[i]=a[i]+'a'-'A';
            //a[i]=tolower(a[i]);
}

void sort_substr(char a[]){
      int i,j;
      int upp=0,low=0;
      char t;
      while(1){
          for(upp=low;a[upp]&&a[upp]!=' ';upp++);
          for(i=low;i<upp-1;i++)
             for(j=i+1;j<=upp-1;j++)
                    if(a[j]<a[i]){
                          t=a[i];
                   a[i]=a[j];
                   a[j]=t;
                    }
          if(a[upp]==0)break;
          low=upp+1;
      }
}
```

```
int main(){
        char s[20][100];
        int n,i;
        cin>>n;
        cin.get();
        for(i=0;i<n;i++){
                cin.getline(s[i],99);
                upp_to_low(s[i]);
                sort_substr(s[i]);
        }
        for(i=0;i<n;i++)cout<<s[i]<<endl;
}
```

# 缩略词

缩略词是单词序列中每个单词的首字母的大写形式，但不包含of, and, the等词。比如nanjing university of science and technology，缩略词为NUST，Nanjing University of Posts and Telecommunications，缩略词为NUPT。

本程序输入一组单词序列（序列长度不超过100，其中至少有一个单词，多个单词之间由一个或多个空格分割），输出缩略词（缩略词长度不超过20）。

(1)编写一个函数char * getAbbr(const char * str, char * result);，其中参数str是输入的原字符串，参数result是缩略词的串，函数返回缩略词的首地址以方便调用。

若运行程序段

```
char res[20];
cout<<getAbbr("nanjing  university of science and technology", res);
```

res中应保存字符串NUST，且输出NUST。

或者编写一个函数string getAbbr(const string str);，其中参数str是输入的原字符串，返回缩略词的字符串。

(2)编写main函数，反复输入单词序列，调用上面函数并显示结果，直到输入字符串exit为止。

```
#include<iostream>
#include<string>
#include<sstream>
using namespace std;
string getAbbr(const string str){
	string res="";
	stringstream input(str);
	string word;
	while (input >> word) {
		if (word == "of" || word == "and" || word == "the")
			continue;
		res += toupper(word.at(0));
	}
	return res;
}
int main(){
	for(;;) {
		char str[100];  //string str;
		cin.getline(str, 100);
		string str2(str);    //getline(cin,str);
		if (str2 == "exit")break;
		cout<<getAbbr(str2)<<endl;
}
```

```
#include<iostream>
#include<cstring>
using namespace std;
//原串为形参str,返回string去缩写串
//空格分割的多个词
char * getAbbr(const char * str, char * result) {
   int reslen=0;
   for(int i=0; i< strlen(str); i++) {
      char word[30];
      int j=0;
      while(str[i] != ' ' && str[i] != '\0')
         word[j++] = str[i++];
      if(j>0){
         word[j] = '\0';
          if (strcmp(word,"of")!=0 && strcmp(word,"and")!=0 &&
strcmp(word,"the")!=0) {
            result[reslen++] = toupper(word[0]);
         }
      }
   }
   result[reslen] = '\0';
   return result;
}
```

```
int main() {
    //反复输入测试，直到输入exit为止
    /*
    for(;;) {
        char str[100];
        cin.getline(str, 100);
        if (strcmp(str,"exit")==0)
            break;
        char res[30];
        cout<<getAbbr(str,res)<<endl;
    }
}
```

# 小结

- 一个数组是由相同类型的一组变量组成的一个有序集合。
- 数组中的每个变量称为一个元素，所有元素共用一个变量名，就是数组的名字。
- 数组中的每个元素都有一个序号，称为下标。访问一个元素可用数组名加下标来实现。
- 定义一个数组有3个要素：类型、名称与长度(即大小)。一个数组占用一块连续存储空间，字节大小为n*sizeof(<元素类型>)，其中n为数组的长度。一个数组的长度必须在定义时确定，而且在执行过程中不能改变。
- 通过表达式下标访问数组元素时，注意下标越界。
- 两个数组之间不能直接赋值，也不宜使用关系运算符。
- 数组具有多种实际用途，如多项式计算、排序(如选择排序、冒泡排序等)、查找(如二分查找)等。